

利用格点 QCD 研究 部分子分布函数 ——传播子的高性能求解

兰州大学

核科学与技术学院

2021级核物二班

张鑫

代码托管:

<https://github.com/zhangxin8069/qcu.git>; <https://gitee.com/zhangxin8069/qcu.git>;
<https://github.com/zhangxin8069/PyQCU.git>; <https://gitee.com/zhangxin8069/PyQCU.git>;

代码运行环境:

SNSC;COMPUTER-4060;X99;

电话:

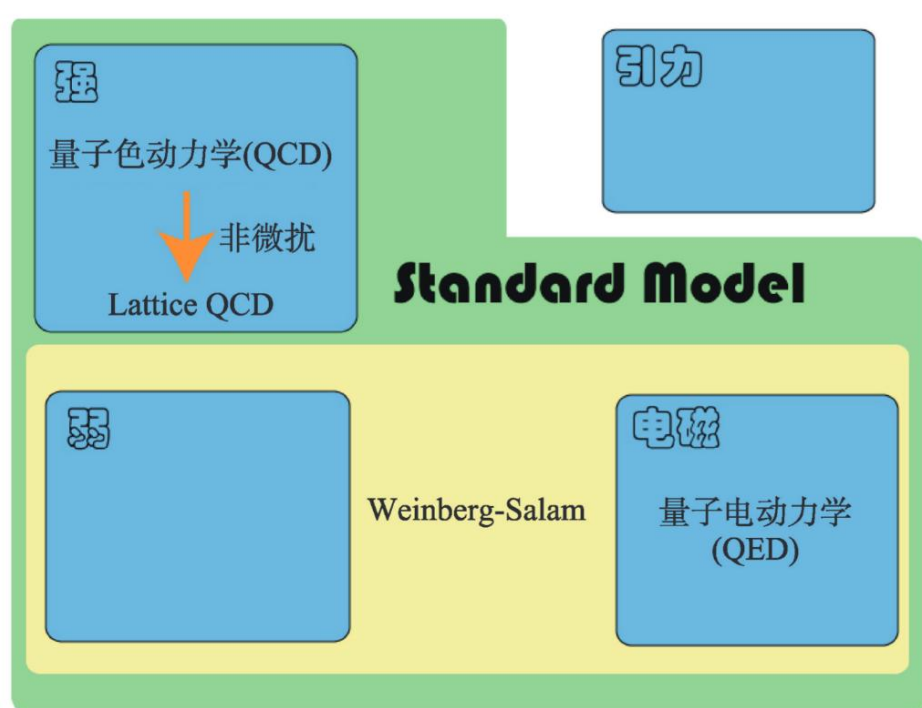
15572348069;

邮箱:

zhangxin2021@lzu.edu.cn;

zhangxin9542@gmail.com;

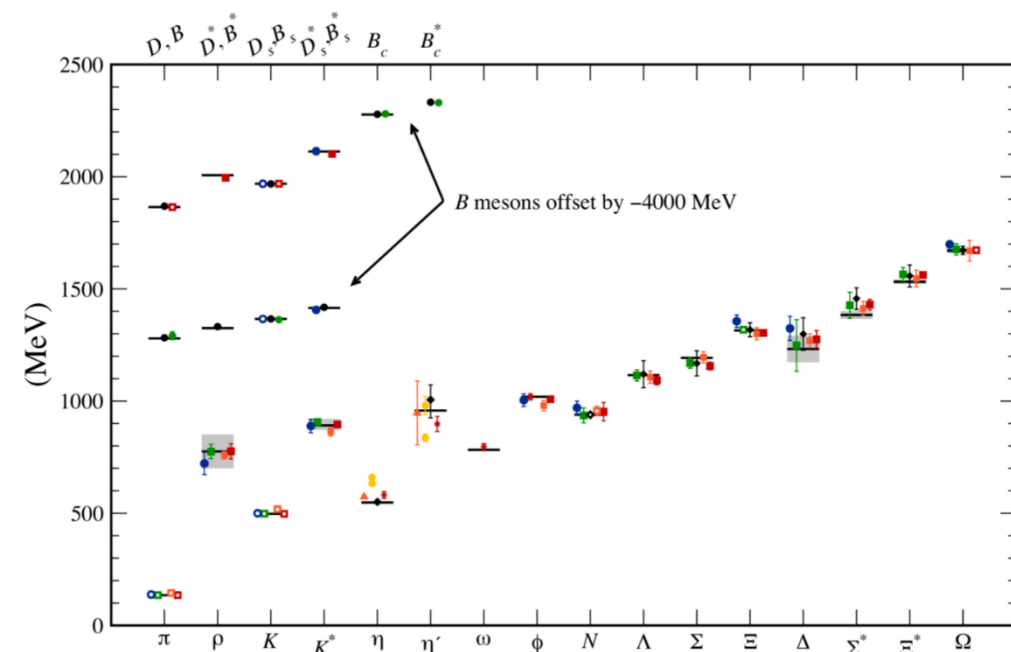
zhangxin8069@qq.com;

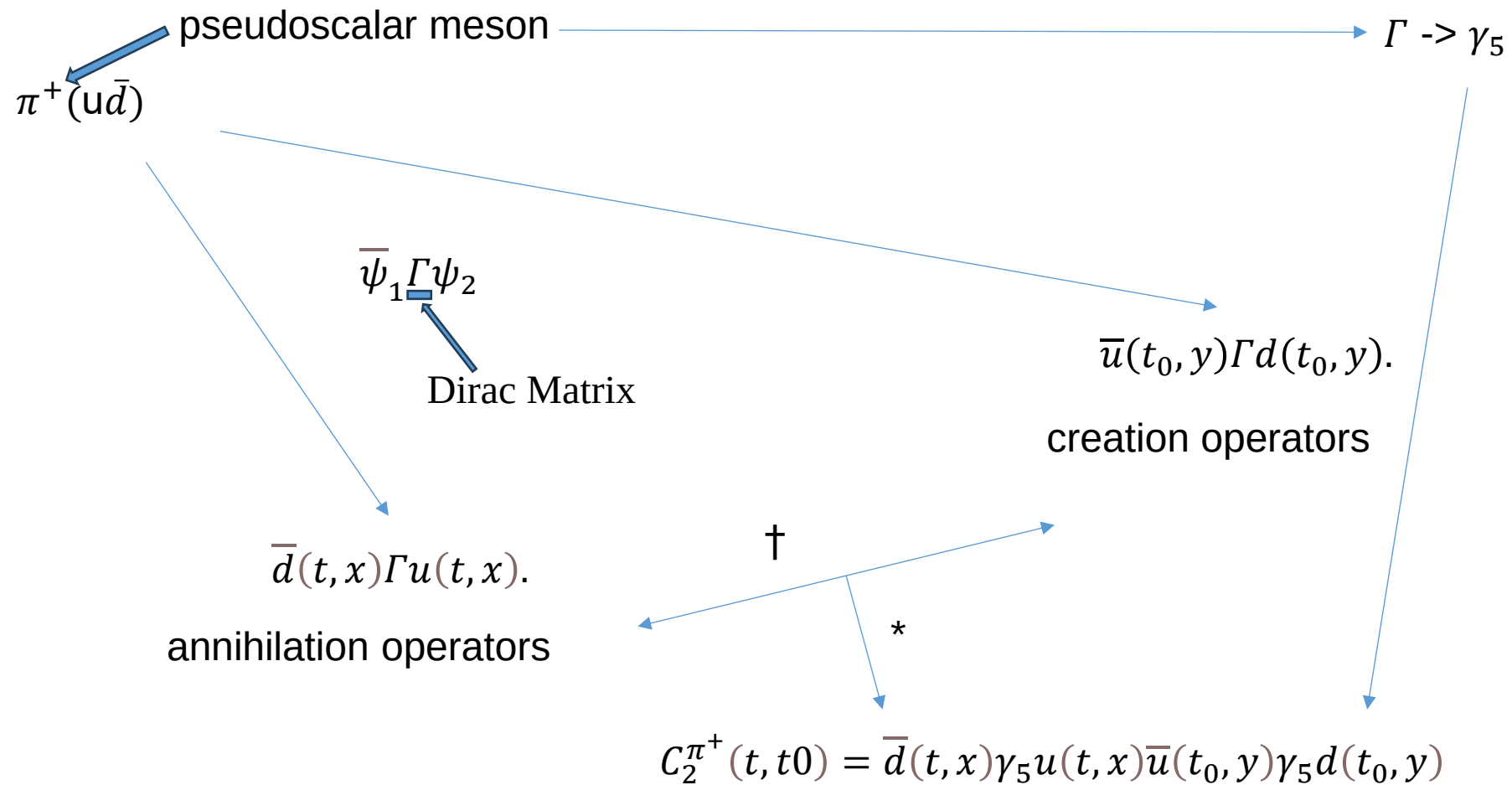


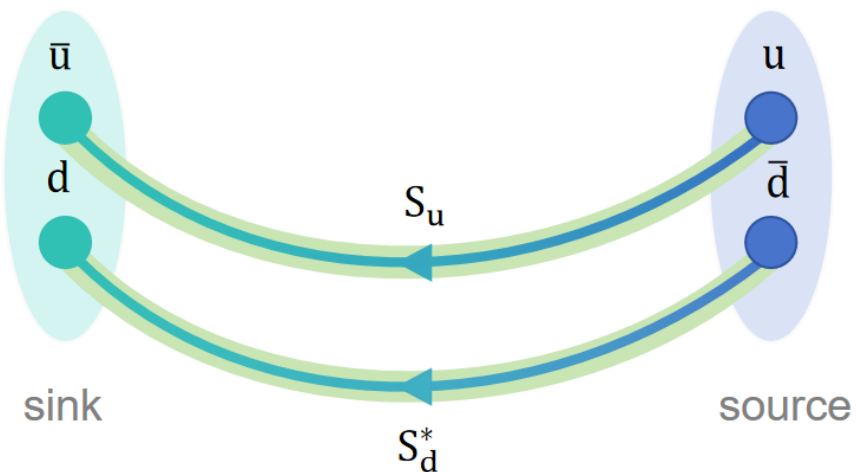
格点量子色动力学（Lattice Quantum Chromodynamics，简称格点 QCD）作为目前较为成熟的研究量子色动力学非微扰问题的基准性数值方法，在强相互作用物理研究中具有不可忽略的作用。**本课题聚焦于该方法的核心应用场景——部分子分布函数的数值研究，并针对其中的关键计算瓶颈即传播子的高效求解展开攻关。**从数学形式上看，该问题可归结为具有特殊结构的大规模稀疏复系数矩阵线性方程组的求解，其维度规模在传统通用计算框架下已超出可处理范围，亟需结合其数学特性设计专用的高性能并行求解算法，这正是本课题研究的核心目标与技术突破点。

关键词：

**理论物理，
第一性原理计算，
LQCD，高性能算法，
CUDA并行软件**







$$\overline{d_b(t_0, \mathbf{y}) d_a(t, \mathbf{x})} = \left(\mathcal{M}^{(d)}[U_\mu] \right)^{-1}_{b, t_0, \mathbf{y}; a, t, \mathbf{x}},$$

$$\overline{u_a(t, \mathbf{x}) \bar{u}_b(t_0, \mathbf{y})} = \left(\mathcal{M}^{(u)}[U_\mu] \right)^{-1}_{a, t, \mathbf{x}; b, t_0, \mathbf{y}},$$

(5.15)

$$C_2^{\pi^+}(t, t_0) = \overline{d}(t, x) \gamma_5 u(t, x) \overline{u}(t_0, y) \gamma_5 d(t_0, y)$$

Wick contraction

A property of Wick's contraction :
each exchange of quark fields produces a phase difference of $\pi \rightarrow '-'$

$$\begin{aligned} C_2^{\pi^+}(t, t_0) &= -\gamma_5 u(t, x) \overline{u}(t_0, y) \gamma_5 d(t_0, y) \overline{d}(t, x) \\ &= -\gamma_5 S^u(t, t_0, x, y) \gamma_5 S^{*,d}(t_0, t, y, x) \\ &= -\gamma_5 S^q(x, y) \gamma_5 S^q(y, x) \end{aligned}$$

其中 S 被称为传播子，其具体表现为费米子矩阵的逆 $\mathcal{M}^{-1}$ [1]

Dslash - wilson

$$\kappa = \frac{1}{2(m_0 a + 4)}$$

Gamma Matrix →

$$\gamma_0(\gamma_x) = \begin{pmatrix} 0 & 0 & 0 & i \\ 0 & 0 & i & 0 \\ 0 & -i & 0 & 0 \\ -i & 0 & 0 & 0 \end{pmatrix}$$

$$\gamma_1(\gamma_y) = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}$$

$$\gamma_2(\gamma_z) = \begin{pmatrix} 0 & 0 & i & 0 \\ 0 & 0 & 0 & -i \\ -i & 0 & 0 & 0 \\ 0 & i & 0 & 0 \end{pmatrix}$$

$$\gamma_3(\gamma_t) = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

↓

Wilson Dslash



$$2\kappa\mathcal{M}[U_\mu]_{x,y} = \sigma_{x,y} - \kappa \sum_{\mu=1}^4 [(1 - \gamma_\mu)U_\mu(x)\sigma_{x+\mu,y} + (1 + \gamma_\mu)U_\mu^\dagger(x - \mu)\sigma_{x-\mu,y}]$$

↓ †(γ_μ is Hermitic)

$$\sigma_{x,y} - \kappa \sum_{\mu=1}^4 [(1 - \gamma_\mu)U_\mu^\dagger(x)\sigma_{y+\mu,x} + (1 + \gamma_\mu)U_\mu(x - \mu)\sigma_{y-\mu,x}]$$



(Wilson Dslash)^{Dagger}

Trick – SU(3) for Dslash

Trading computational consumption for bandwidth consumption.

```
324 #define give_u_laplacian(U, tmp_U, lat_tzyx) \  
325 { \  
326     for (int i = 0; i < _LAT_CC_; i++) \  
327     { \  
328         U[i] = tmp_U[i * _LAT_D_ * lat_tzyx]; \  
329     } \  
330 }
```



```
314 #define _give_u_laplacian(U, tmp_U, lat_tzyx) \  
315 { \  
316     for (int i = 0; i < _LAT_2C_; i++) \  
317     { \  
318         U[i] = tmp_U[i * _LAT_D_ * lat_tzyx]; \  
319     } \  
320     U[6] = (U[1] * U[5] - U[2] * U[4]).conj(); \  
321     U[7] = (U[2] * U[3] - U[0] * U[5]).conj(); \  
322     U[8] = (U[0] * U[4] - U[1] * U[3]).conj(); \  
323 }
```

Dslash - clover

$a = 1$ (always ignore)

$r = 1$ (in code written as *coeff_r*)

$C_{SW} = 1$ (in code written as *coeff_t*)

$$\kappa = \frac{1}{2m_q a + 8r}$$

or just define($m = -3.5$) :

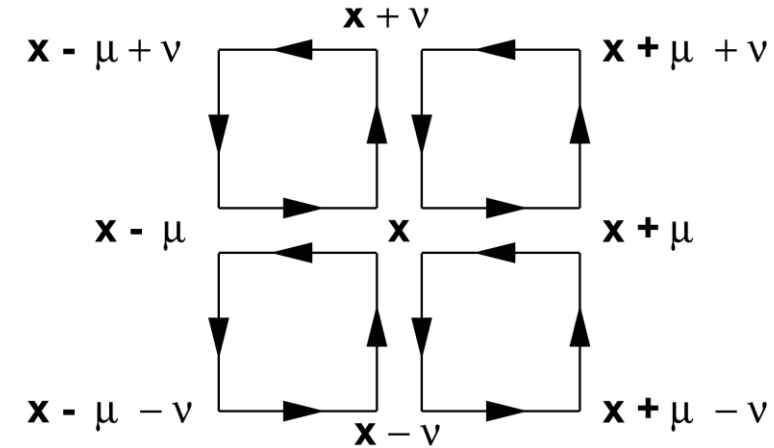
$\kappa = 1$ (in code written as *kappa*)

$$M_{x,y}^W[U]a = \delta_{xy} - \kappa \sum_{\mu} [(r - \gamma_{\mu})U_{x,\mu}\delta_{x,y-\mu} + (r + \gamma_{\mu})U_{x-\mu,\mu}^{\dagger}\delta_{x,y+\mu}]$$

$$- \frac{iaC_{SW}\kappa r}{4} \sigma_{\mu\nu} F_{\mu\nu} \delta_{x,y}$$

← Clover Part

Dslash - clover



$$\sigma_{\mu\nu} F_{\mu\nu} = 2\gamma_\mu \gamma_\nu \frac{1}{4} \sum_p \frac{1}{2} [U_p(x) - U_p^\dagger(x)]$$

$$= \frac{1}{4} \gamma_\mu \gamma_\nu [$$

$$\begin{aligned} U_1 &= u(x, \mu) u(x + \mu, \nu) u^\dagger(x + \nu, \mu) u^\dagger(x, \nu) \\ U_2 &= u(x, \nu) u^\dagger(x - \mu + \nu, \mu) u^\dagger(x - \mu, \nu) u(x - \mu, \mu) \\ U_3 &= u^\dagger(x - \mu, \mu) u^\dagger(x - \mu - \nu, \nu) u(x - \mu - \nu, \mu) u(x - \nu, \nu) \\ U_4 &= u^\dagger(x - \nu, \nu) u(x - \nu, \mu) u(x - \nu + \mu, \nu) u^\dagger(x, \mu) \end{aligned}$$

$$u(x, \mu) u(x + \mu, \nu) u^\dagger(x + \nu, \mu) u^\dagger(x, \nu)$$

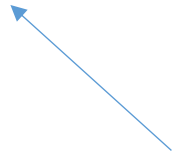
$$+ u(x, \nu) u^\dagger(x - \mu + \nu, \mu) u^\dagger(x - \mu, \nu) u(x - \mu, \mu)$$

$$+ u^\dagger(x - \mu, \mu) u^\dagger(x - \mu - \nu, \nu) u(x - \mu - \nu, \mu) u(x - \nu, \nu) \quad \text{Before the '['} \quad \downarrow$$

$$+ u^\dagger(x - \nu, \nu) u(x - \nu, \mu) u(x - \nu + \mu, \nu) u^\dagger(x, \mu) - BEFORE^\dagger]$$

Dslash - clover

When using clover dslash in practice, you just need to multiply the clover term by wilson dslash, so the following examples will only give wilson's example for the sake of simplicity.



```
6 void applyCloverDslashQcu(long long _fermion_out, long long _fermion_in, long long _gauge, long long _set_ptrs, long long _params)
7 {
8     cudaDeviceSynchronize();
9     void *fermion_out = (void *)_fermion_out;
10    void *fermion_in = (void *)_fermion_in;
11    void *gauge = (void *)_gauge;
12    void *set_ptrs = (void *)_set_ptrs;
13    void *params = (void *)_params;
14    int set_index = static_cast<int *>(params)[_SET_INDEX_];
15    LatticeSet<T> *set_ptr = static_cast<LatticeSet<T> *>((void *) (static_cast<long long *>(set_ptrs)[set_index])); // define for apply_clover_dslash
16    // dptzyxcc2ccdptzyx<T>(gauge, &set);
17    // tzyxsc2sctzyx<T>(fermion_in, &set);
18    // tzyxsc2sctzyx<T>(fermion_out, &set);
19    LatticeWilsonDslash<T> _wilson_dslash;
20    LatticeCloverDslash<T> _clover_dslash;
21    _wilson_dslash.give(set_ptr);
22    _clover_dslash.give(set_ptr);
23    _clover_dslash.init();
24    {
25        // wilson dslash
26        _wilson_dslash.run_test(fermion_out, fermion_in, gauge);
27    }
28    {
29        // make clover
30        _clover_dslash.make(gauge);
31    }
32    {
33        // inverse clover
34        _clover_dslash.inverse();
35    }
36    {
37        // give clover
38        _clover_dslash.give(fermion_out);
39    }
40    // ccdptzyx2dptzyxcc<T>(gauge, &set);
41    // sctzyx2tzyxsc<T>(fermion_in, &set);
42    // sctzyx2tzyxsc<T>(fermion_out, &set);
43    _clover_dslash.end();
44    cudaDeviceSynchronize();
45 }
```

Inverse

In the calculation of LQCD, we often need to solve $D\text{slash}(\text{dest})=\text{src}$, this process is conventionally called inversion, and the commonly used solving algorithms are the following three:

算法 1 Conjugate gradient

```
1: procedure CG( $A, b$ )
2:    $x \leftarrow 0$ 
3:    $r \leftarrow b - Ax$ 
4:   If converge return  $x$ 
5:    $p \leftarrow r$ 
6:   while not reach max iteration do
7:      $\alpha \leftarrow \frac{\langle r, r \rangle}{\langle p, Ap \rangle}$ 
8:      $x \leftarrow x + \alpha p$ 
9:      $r' \leftarrow r - \alpha Ap$ 
10:    If converge return  $x$ 
11:     $\beta \leftarrow \frac{\langle r', r' \rangle}{\langle r, r \rangle}$ 
12:     $p \leftarrow r' + \beta p$ 
13:     $r \leftarrow r'$ 
14:  end while
15: end procedure
```

CG

(Conjugate Gradient Method)

ps:

In order to preserve the positive
definite condition, give
 $D\text{slash}' \rightarrow D\text{slash}^{\dagger} * D\text{slash}$

BICGSTAB
(Biconjugate gradient
stabilized method)

More balanced.



Easiest to implement and slowest.

MG
(Multigrid Method)



Hardest to implement, fastest.

算法 3 Biconjugate gradient stabilized

```
1: procedure BiCGSTAB( $A, b$ )
2:    $x \leftarrow 0$ 
3:    $r \leftarrow r_0 \leftarrow b - Ax$ 
4:   If converge return  $x$ 
5:    $p \leftarrow r$ 
6:   while not reach max iteration do
7:      $\alpha \leftarrow \frac{\langle r_0, r \rangle}{\langle r_0, Ap \rangle}$ 
8:      $x \leftarrow x + \alpha p$ 
9:      $r' \leftarrow r - \alpha Ap$ 
10:    If converge return  $x$ 
11:     $t \leftarrow Ar$ 
12:     $\omega \leftarrow \frac{\langle t, r \rangle}{\langle t, t \rangle}$ 
13:     $x \leftarrow x + \omega r'$ 
14:     $r' \leftarrow r' - \omega Ar'$ 
15:    If converge return  $x$ 
16:     $\beta \leftarrow \frac{\langle r', r' \rangle}{\langle r, r \rangle}$ 
17:     $p \leftarrow r' + (\alpha\beta/\omega)p - \alpha\beta Ap$ 
18:     $r \leftarrow r'$ 
19:  end while 11
20: end procedure
```

Trick - even-odd preprocessing for Inverse

Although the computational consumption of a single loop is unchanged, the memory occupation is reduced to half of the original, the overall convergence times are reduced, and the efficiency is significantly improved.

$$A_{ee}x_e - \kappa D_{eo}x_o = b_e$$

$$(A_{oo} - \kappa^2 D_{oe}A_{ee}^{-1}D_{eo})x_o = \kappa D_{oe}A_{ee}^{-1}b_e + b_o$$

origin one : $\mathcal{M}x = b$

$$\begin{pmatrix} 1 - \kappa T_{ee} & -\kappa D_{eo} \\ -\kappa D_{oe} & 1 - \kappa T_{oo} \end{pmatrix} \begin{pmatrix} x_e \\ x_o \end{pmatrix} = \begin{pmatrix} b_e \\ b_o \end{pmatrix}$$

$$A = 1 + T$$

in this case, $T = 0$, so $A = 1$

so,

$$\begin{pmatrix} 1 & -\kappa D_{eo} \\ -\kappa D_{oe} & 1 \end{pmatrix} \begin{pmatrix} x_e \\ x_o \end{pmatrix} = \begin{pmatrix} b_e \\ b_o \end{pmatrix}$$

$$x_e - \kappa D_{eo}x_o = b_e$$

$$x_o - \kappa D_{oe}x_e = b_o$$

so,

$$x_e - \kappa D_{eo}x_o = b_e$$

$$x_o - \kappa D_{oe}(\kappa D_{eo}x_o + b_e) = b_o$$

then

$$x_e - \kappa D_{eo}x_o = b_e$$

$$x_o - \kappa^2 D_{oe}(D_{eo}x_o) = \kappa D_{oe}b_e + b_o$$

so,

$$\text{just solve } x_o - \kappa D_{oe}(\kappa D_{eo}x_o) = \kappa D_{oe}b_e + b_o$$

$$\text{then will easily get } x_e \text{ by } x_e - \kappa D_{eo}x_o = b_e$$

so,

give Dslash :

$$tmp = D_{eo}src_o$$

$$dest_o = src_o - \kappa D_{oe}(\kappa D_{eo}src_o) = src_o - \kappa^2 D_{oe}tmp$$

give b:

$$b_e = anw_e - \kappa D_{eo}anw_o$$

$$b_o = anw_o - \kappa D_{oe}anw_e$$

$$b'_o = b_o + \kappa D_{oe}b_e$$

so,

$$Dslash(x_o) = b'_o$$

then get x_o by BistabCg,

This comes at the cost of a bit more overall code complexity.....

finally get x_e by $b_e + \kappa D_{eo}x_o$

Trick - even-odd preprocessing for Inverse

$$A_{ee}x_e - \kappa D_{eo}x_o = b_e$$

$$(A_{oo} - \kappa^2 D_{oe}A_{ee}^{-1}D_{eo})x_o = \kappa D_{oe}A_{ee}^{-1}b_e + b_o$$

origin one : $\mathcal{M}x = b$

$$\begin{pmatrix} 1 - \kappa T_{ee} & -\kappa D_{eo} \\ -\kappa D_{oe} & 1 - \kappa T_{oo} \end{pmatrix} \begin{pmatrix} x_e \\ x_o \end{pmatrix} = \begin{pmatrix} b_e \\ b_o \end{pmatrix}$$

$$A = 1 + T$$

in this case, $T = 0$, so $A = 1$

so,

$$\begin{pmatrix} 1 & -\kappa D_{eo} \\ -\kappa D_{oe} & 1 \end{pmatrix} \begin{pmatrix} x_e \\ x_o \end{pmatrix} = \begin{pmatrix} b_e \\ b_o \end{pmatrix}$$

$$x_e - \kappa D_{eo}x_o = b_e$$

$$x_o - \kappa D_{oe}x_e = b_o$$

so,

$$x_e - \kappa D_{eo}x_o = b_e$$

$$x_o - \kappa D_{oe}(\kappa D_{eo}x_o + b_e) = b_o$$

then

$$x_e - \kappa D_{eo}x_o = b_e$$

$$x_o - \kappa^2 D_{oe}(D_{eo}x_o) = \kappa D_{oe}b_e + b_o$$

定义 $A = 1 + T$,

$$\begin{pmatrix} A_{ee} & -\kappa D_{eo} \\ -\kappa D_{oe} & A_{oo} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ -\kappa D_{oe}A_{ee}^{-1} & 1 \end{pmatrix} \begin{pmatrix} A_{ee} & 0 \\ 0 & A_{oo} - \kappa^2 D_{oe}A_{ee}^{-1}D_{eo} \end{pmatrix} \begin{pmatrix} 1 & -\kappa A_{ee}^{-1}D_{eo} \\ 0 & 1 \end{pmatrix}$$

左边下三角矩阵的逆可以轻松得到,

$$\begin{pmatrix} 1 & 0 \\ -\kappa D_{oe}A_{ee}^{-1} & 1 \end{pmatrix}^{-1} = \begin{pmatrix} 1 & 0 \\ \kappa D_{oe}A_{ee}^{-1} & 1 \end{pmatrix}.$$

考虑将 x 和 b 也分成奇偶两部分, 初始的线性方程组 $\mathcal{M}x = b$ 变为

$$\begin{cases} A_{ee}x_e - \kappa D_{eo}x_o = b_e, \\ (A_{oo} - \kappa^2 D_{oe}A_{ee}^{-1}D_{eo})x_o = \kappa D_{oe}A_{ee}^{-1}b_e + b_o. \end{cases}$$

Inverse - cg

The dslash' of cg.



```

256 void _run()
257 {
258     checkCudaErrors(cudaStreamSynchronize(set_ptr->stream));
259     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_a]));
260     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_b]));
261     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_c]));
262     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_d]));
263     {
264         // rho = <r, r>;
265         _dot(r, r, _rho_, _a_);
266     }
267     for (int loop = 0; loop < set_ptr->max_iter(); loop++)
268     {
269         {
270             // v = A * p;
271             _wilson_dslash(v, p, gauge);
272         }
273         {
274             // rho_prev = rho = <r, r>;
275             cg_give_1rho_prev<T><<<1, 1, 0, set_ptr->streams[_a]>>>(
276                 device_vals);
277         }
278         checkCudaErrors(cudaStreamSynchronize(set_ptr->stream));
279         {
280             // tmp0 = <p, Ap> = <p, v>;
281             _dot(p, v, _tmp0_, _b_);
282         }
283         checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_a]));
284         {
285             // alpha = <r, r>/<p, Ap> = rho_prev/tmp0;
286             cg_give_1alpha<T><<<1, 1, 0, set_ptr->streams[_b]>>>(device_vals)
287         }
288         checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_b]));
289         {
290             // r_tilde[i] = r[i] - v * alpha;
291             // r => r_tilde
292             cg_give_r_tilde<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
293                 set_ptr->streams[_c]>>>(r, v, device_vals);
294         }

```

```

295     {
296         // x_o[i] = x_o[i] + p * alpha;
297         cg_give_x_o<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
298             set_ptr->streams[_d]>>>(x_o, p, device_vals);
299     }
300     {
301         // rho = <r_tilde, r_tilde>;
302         _dot(r, r, _rho_, _c_);
303     }
304     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_c]));
305     {
306         // beta = <r_tilde, r_tilde>/<r, r> = rho/rho_prev;
307         cg_give_1beta<T><<<1, 1, 0, set_ptr->streams[_d]>>>(device_vals)
308     }
309     {
310         // p[i] = r_tilde[i] + p[i] * beta
311         cg_give_p<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
312             set_ptr->streams[_d]>>>(p, r, device_vals);
313     }
314     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_d]));
315     {
316         #ifndef PRINT_MULTI_GPU_WILSON_CG
317             print_vals(loop);
318             std::cout << "###RANK:" << set_ptr->host_params[_NODE_RANK_]
319                 << "###LOOP:" << loop
320                 << "###Residual:" << host_vals[_rho_prev_]._data.x
321                 << std::endl;
322         #endif
323     }
324     if ((host_vals[_rho_]._data.x < set_ptr->tol() || loop == set_ptr->max_iter() - 1))
325     {
326         std::cout << "###RANK:" << set_ptr->host_params[_NODE_RANK_]
327             << "###LOOP:" << loop << "###Residual:" << host_vals[_rho_]
328             << std::endl;
329         break;
330     }
331 }
332 checkCudaErrors(cudaStreamSynchronize(set_ptr->stream));
333 checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_a]));
334 checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_b]));
335 checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_c]));
336 checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_d]));

```

```

111 void _wilson_dslash_no_dag(void *fermion_out, void *fermion_in, void *gauge)
112 {
113     // src_o=set_ptr->kappa()*2*dslash_oe(dslash_oe(src_o))
114     wilson_dslash.run_oe(device_vec0, fermion_in, gauge);
115     wilson_dslash.run_oe(device_vec1, device_vec0, gauge);
116     cg_give_dest_o<T><<<set_ptr->gridDim, set_ptr->blockDim, 0, set_ptr->stream>>>(
117         fermion_out, fermion_in, device_vec1, set_ptr->kappa(), device_vals);
118 }
119 void _wilson_dslash_dag(void *fermion_out, void *fermion_in, void *gauge)
120 {
121     // src_o=set_ptr->kappa()*2*dslash_oe(dslash_oe(src_o))
122     wilson_dslash.run_oe_dag(device_vec0, fermion_in, gauge);
123     wilson_dslash.run_oe_dag(device_vec1, device_vec0, gauge);
124     cg_give_dest_o<T><<<set_ptr->gridDim, set_ptr->blockDim, 0, set_ptr->stream>>>(
125         fermion_out, fermion_in, device_vec1, set_ptr->kappa(), device_vals);
126 }
127 void _wilson_dslash(void *fermion_out, void *fermion_in, void *gauge)
128 {
129     _wilson_dslash_no_dag(device_vec2, fermion_in, gauge);
130     _wilson_dslash_dag(fermion_out, device_vec2, gauge);
131 }

```

← The main body of cg.

Inverse - bicgstab

The dslash' of bicgstab.



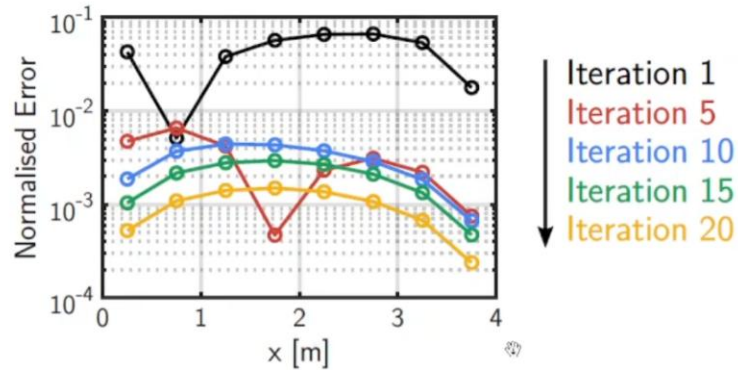
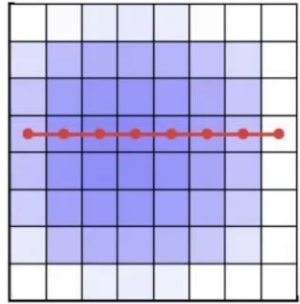
```
void _wilson_dslash(void *fermion_out, void *fermion_in, void *gauge)
{
    // src_o-set_ptr->kappa()*2*dslash_oe(dslash_oe(src_o))
    wilson_dslash.run_oe(device_vec0, fermion_in, gauge);
    wilson_dslash.run_oe(device_vec1, device_vec0, gauge);
    bistabcg_give_dest_o<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
        set_ptr->stream>>>{
        fermion_out, fermion_in, device_vec1, set_ptr->kappa(), device_vals);
    }
}
```

← The main body of bicgstab.

```
241 void _run()
242 {
243     checkCudaErrors(cudaStreamSynchronize(set_ptr->stream));
244     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_a]));
245     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_b]));
246     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_c]));
247     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_d]));
248     for (int loop = 0; loop < set_ptr->max_iter(); loop++)
249     {
250         _dot(r_tilde, r, _rho_, _a_);
251         checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_b]));
252         {
253             // beta = (rho / rho_prev) * (alpha / omega);
254             bistabcg_give_1beta<T><<<1, 1, 0, set_ptr->streams[_a]>>>(device_vals);
255         }
256         checkCudaErrors(cudaStreamSynchronize(
257             set_ptr->streams[_a])); // needed, but don't know why.
258         {
259             // rho_prev = rho;
260             bistabcg_give_1rho_prev<T><<<1, 1, 0, set_ptr->streams[_b]>>>(
261                 device_vals);
262         }
263         {
264             // p[i] = r[i] + (p[i] - v[i] * omega) * beta;
265             bistabcg_give_p<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
266                 set_ptr->streams[_a]>>>(p, r, v, device_vals);
267         }
268         checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_a]));
269         _dot(r, r, _norm2_tmp_, _c_);
270         {
271             // v = A * p;
272             _wilson_dslash(v, p, gauge);
273         }
274         checkCudaErrors(cudaStreamSynchronize(set_ptr->stream));
275         _dot(r_tilde, v, _tmp0_, _d_);
276         {
277             // alpha = rho / tmp0;
278             bistabcg_give_1alpha<T><<<1, 1, 0, set_ptr->streams[_d]>>>(device_vals);
279         }
280         checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_d]));
281
282         // s[i] = r[i] - v[i] * alpha;
283         bistabcg_give_s<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
284             set_ptr->streams[_a]>>>(s, r, v, device_vals);
285     }
286     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_a]));
287     {
288         // t = A * s;
289         _wilson_dslash(t, s, gauge);
290     }
291     checkCudaErrors(cudaStreamSynchronize(set_ptr->stream));
292     _dot(t, s, _tmp0_, _c_);
293     _dot(t, t, _tmp1_, _d_);
294     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_c]));
295     {
296         // omega = tmp0 / tmp1;
297         bistabcg_give_1omega<T><<<1, 1, 0, set_ptr->streams[_d]>>>(device_vals);
298     }
299     checkCudaErrors(cudaStreamSynchronize(set_ptr->streams[_d]));
300     {
301         // r[i] = s[i] - t[i] * omega;
302         bistabcg_give_r<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
303             set_ptr->streams[_a]>>>(r, s, t, device_vals);
304     }
305     {
306         // x_o[i] = x_o[i] + p[i] * alpha + s[i] * omega;
307         bistabcg_give_x_o<T><<<set_ptr->gridDim, set_ptr->blockDim, 0,
308             set_ptr->streams[_b]>>>(x_o, p, s, device_vals);
309     }
310     {
311         #ifndef PRINT_MULTI_GPU_WILSON_BISTABCG
312         std::cout << "##RANK:" << set_ptr->host_params[_NODE_RANK_] << "##LOOP:" << loop
313             << "##Residual:" << host_vals[_norm2_tmp_]_data.x
314             << std::endl;
315         #endif
316         if ((host_vals[_norm2_tmp_]_data.x < set_ptr->tol()) ||
317             loop == set_ptr->max_iter() - 1))
318         {
319             std::cout << "##RANK:" << set_ptr->host_params[_NODE_RANK_] << "##LOOP:" << loop
320                 << "##Residual:" << host_vals[_norm2_tmp_] << std::endl;
321             break;
322         }
323     }
}
```

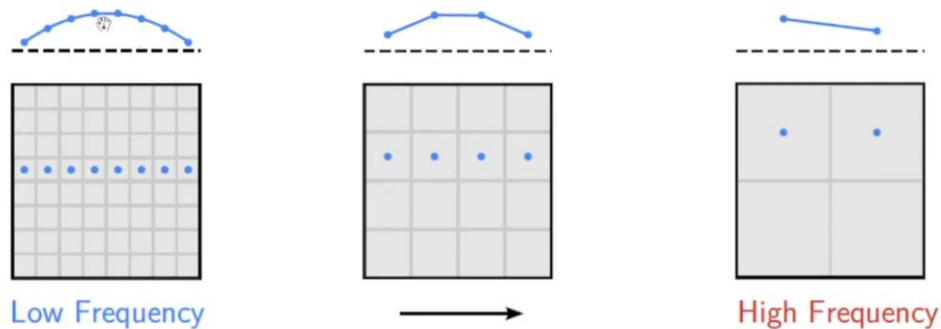
Inverse - mg

Example Problem



- In the first 5 iterations the error field contains both high and low frequencies
- The high spatial frequencies are removed in the first ≈ 10 iterations

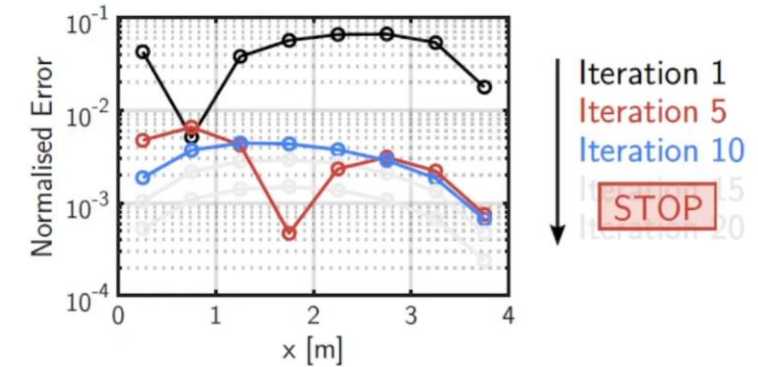
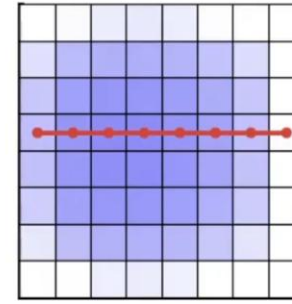
The Multi-Grid Idea



- On a coarser mesh, the **low frequency** components become **high frequency**
- Cells which are far away are now (effectively) connected
- We can remove high frequency components with a few Gauss-Seidel iterations

1

The Multi-Grid Idea

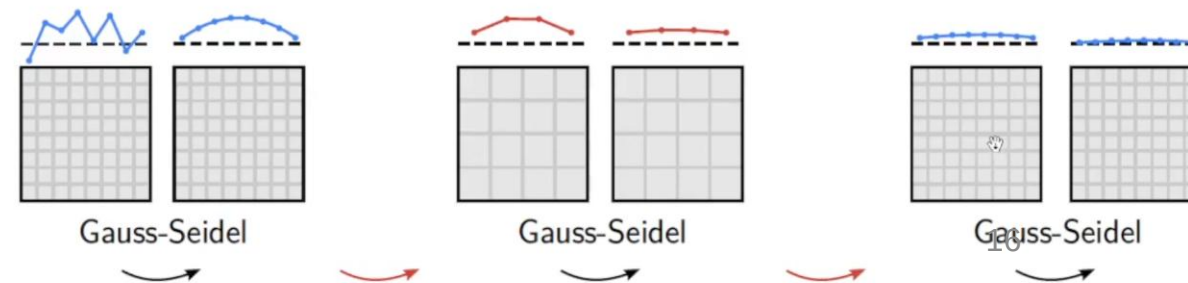


- We can perform a few iterations (say 10) of Gauss-Seidel and then **stop** iterating.
- Gauss-Seidel is then called a **smoother** and not a solver.

2

The Multi-Grid Idea

1. Perform a few iterations to remove the high frequency components (smooth)
2. Move to a coarse mesh
3. Perform a few iterations to remove the low frequency components (smooth)
4. Move back to the fine mesh and perform the final few iterations (solve)



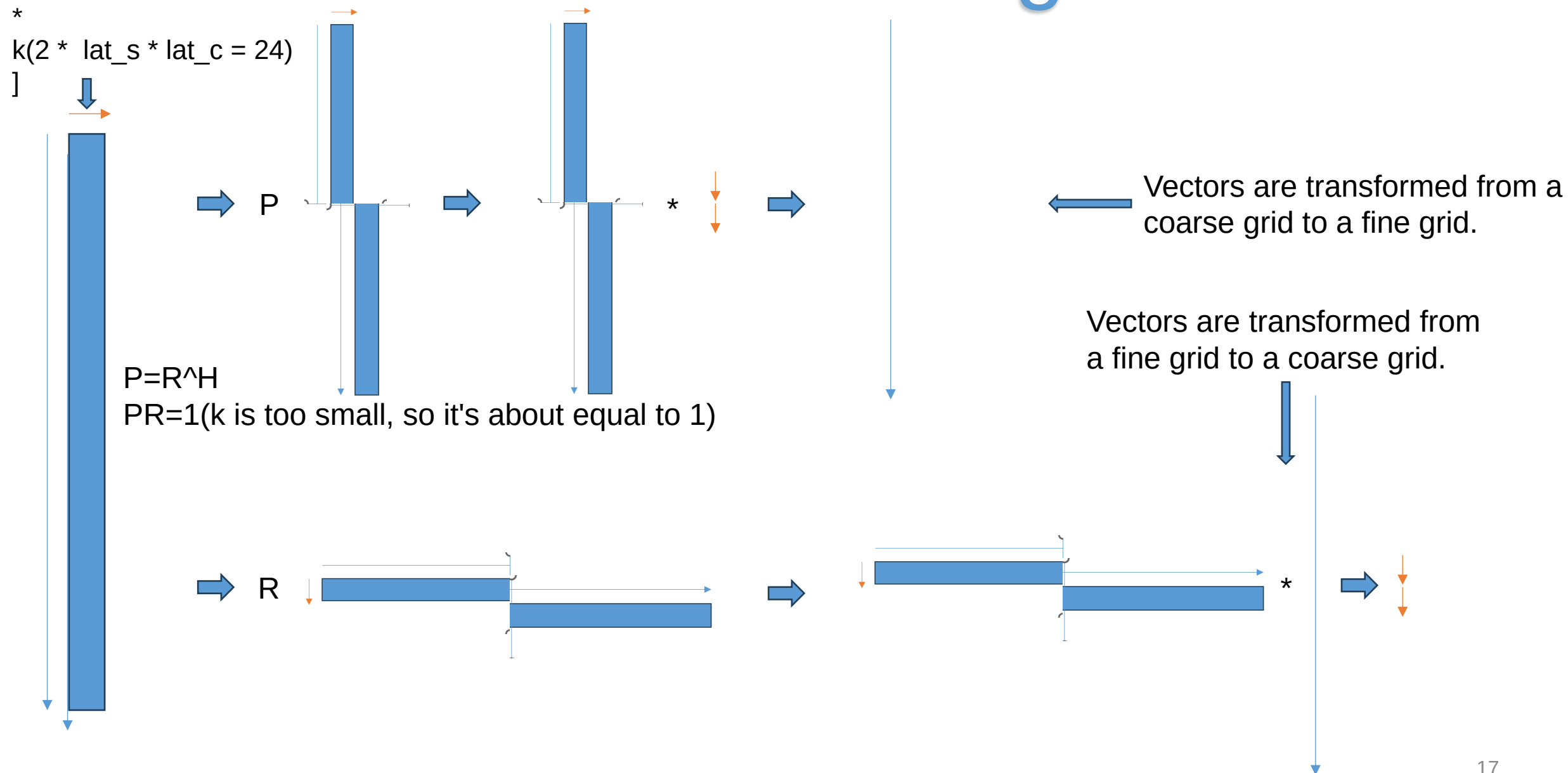
3

4

Eigenvectors of dslash

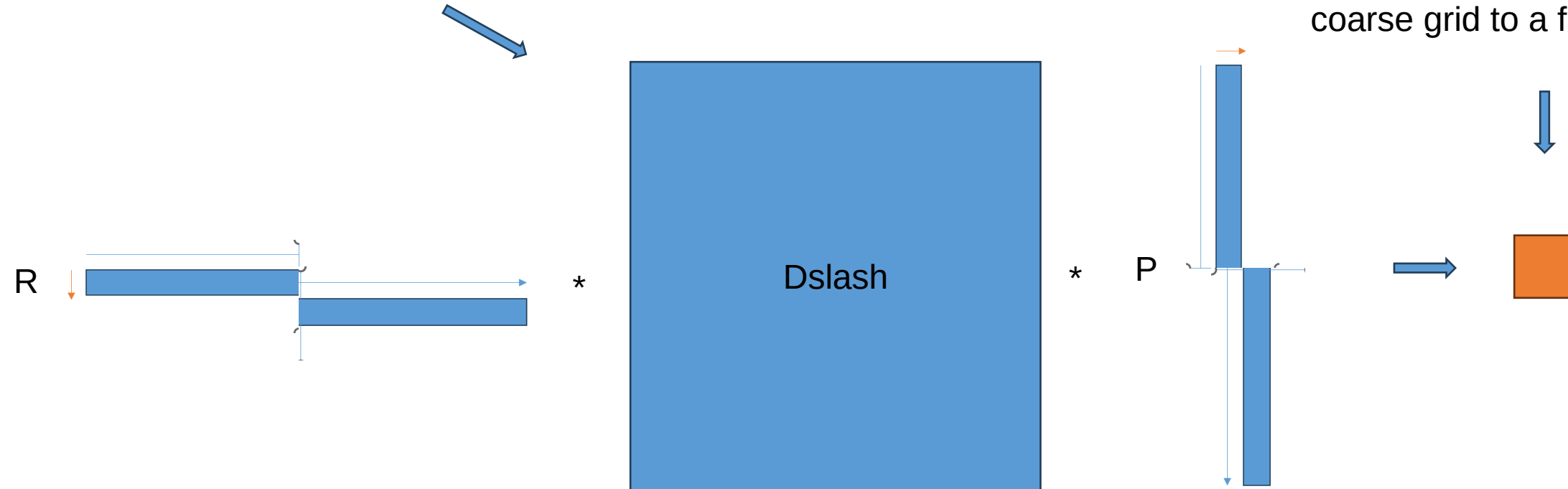
[
V(lat_s * lat_c * lat_t * lat_z * lat_y * lat_x)
*
k(2 * lat_s * lat_c = 24)
]

Inverse - mg



Inverse - mg

Dslash
[
V(lat_s * lat_c * lat_t * lat_z * lat_y * lat_x)
*
V(lat_s * lat_c * lat_t * lat_z * lat_y * lat_x)
]

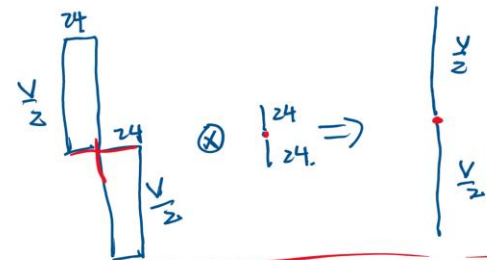


Inverse – mg

Developing...

#2

grid vec $\rightarrow$ origin vec.



$$[(\frac{1}{2} \times 2) \times (24 \times 2)] \otimes [24 \times 2] \Rightarrow [4 \times 1].$$

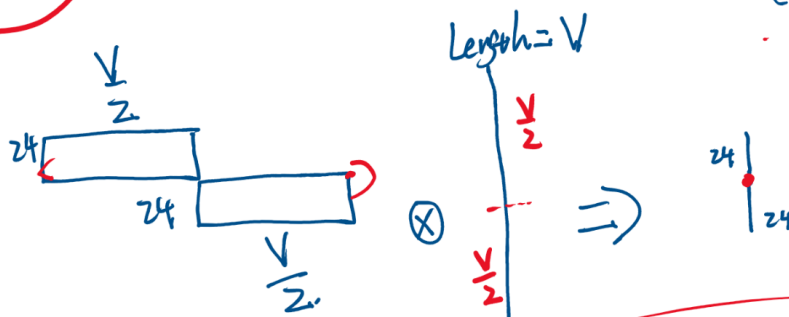
```
Input Array Shape: (24, 4, 3, 32, 32, 32, 16)
Dest Shape: (24, 4, 3, 8, 4, 4, 8, 4, 8, 4, 4)
Input Array Shape: (4, 3, 32, 32, 32, 16)
Dest Shape: (4, 3, 8, 4, 4, 8, 4, 8, 4, 4)
```

```
In [16]: params

array([[ 32,    32,    32,    32, 1048576,    1,    1,
         1,    1,    0,    0,    1,    0, 10000,
         0,    0,    0,    4,    4,    4,    8],
      dtype=int32)
```

MultiGrid - R*vector

#1 origin vec $\rightarrow$ grid vec. $k=24$.
(eigen vector num!)



$$\left[\left[24 \times 2 \right] \times \left(\frac{1}{2} \times 2 \right) \right] \otimes [V \times 1] \Rightarrow \left[(24 \times 2) \times 1 \right]$$

```

1a [H]
    a_src = r_dest

    def p_src(src):
        return contract("src[%d]y0,a[%d]y1", op=src.op_vectors(), src)

    b_dest = p_src(a_src)

2a [H]
    b_dest_stage

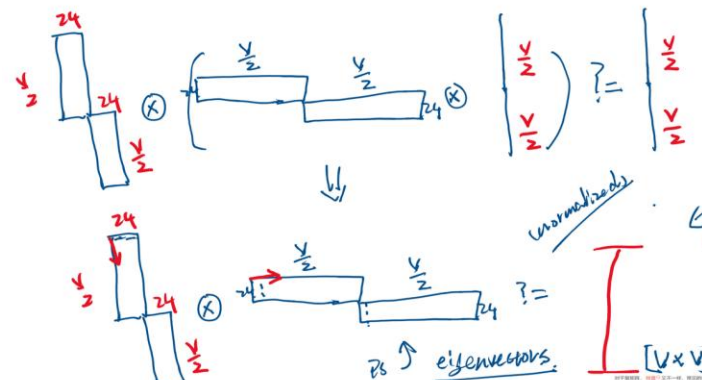
    (A, I, R, A, A, R, A, A, R, A, A)

```

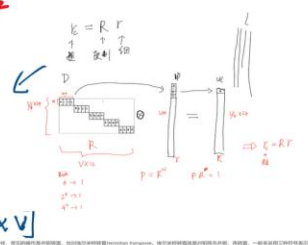
MultiGrid - verify above

#3.

supplementary for above. ↑↑



(normalized)



对于复矩阵, 我们只考虑实对称矩阵, 也就是实对称矩阵 Hermitian Form , 他们和实对称矩阵有类似性质, 例如, 一般实对称三阶矩阵有 3 个实特征值.

1. 第一特征值是 λ^1 , 这时存在实特征向量 $\vec{v}^1$, 使得 $\vec{v}^1$ 和实特征值 λ^1 对应;

2. 第二特征值是 λ^2 , 这时存在实特征向量 $\vec{v}^2$, 使得 $\vec{v}^2$ 和实特征值 λ^2 对应, 那么以下类似:

$$\text{The matrix } A^* \in \mathbb{R}^{n \times n},$$

$$(2.5) \quad A^* = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}$$
[illegible]

V: lat_t * lat_z * lat_y * lat_x

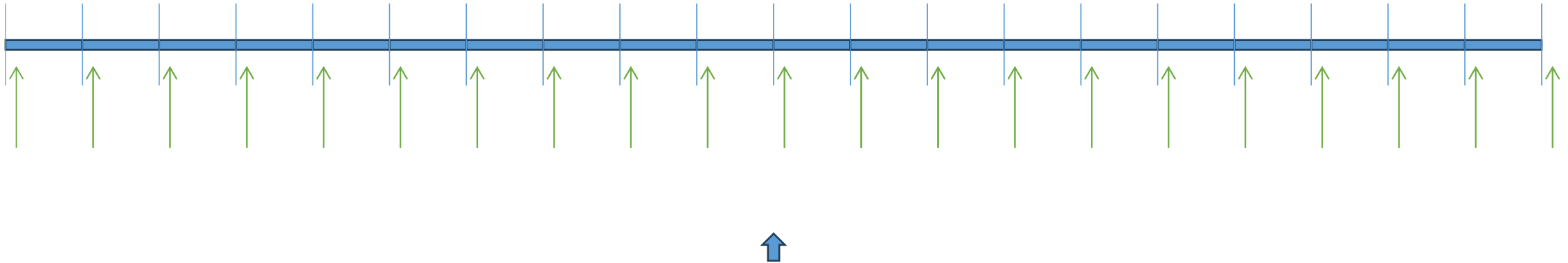
SC: lat_s * lat_c

BLOCK_SIZE: 128 or 256 and so on

GridDim: V/ BLOCK_SIZE

Trick - coalesced in cuda

[V(20)*SC(12)]

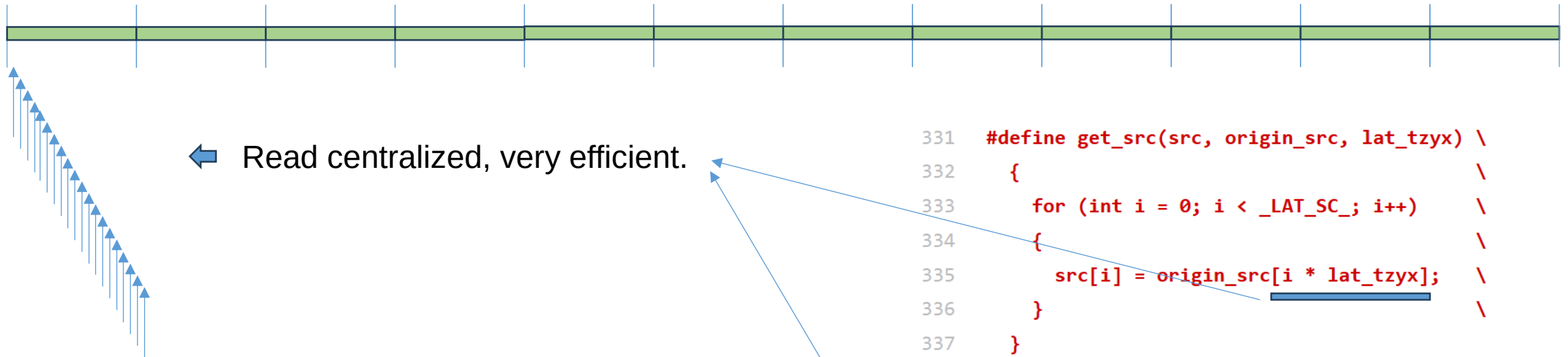


The reads are too scattered and not efficient.

V: lat_t * lat_z * lat_y * lat_x
 SC: lat_s * lat_c
 BLOCK_SIZE: 128 or 256 and so on
 GridDim: V/ BLOCK_SIZE

Trick - coalesced in cuda

[SC(12)*V(20)]



← Read centralized, very efficient.

```

331  #define get_src(src, origin_src, lat_tzyx) \
332  { \
333      for (int i = 0; i < _LAT_SC_; i++) \
334      { \
335          src[i] = origin_src[i * lat_tzyx]; \
336      } \
337  }
```

#####QCU NOTES START#####

0. Required: MPI(e.g. 4.1.2), CUDA(e.g. 12.4), CMAKE(e.g. 3.22.1), GCC(e.g. 11.4.0), HDF5-MPI(e.g. 1.10.7, 'apt install libhdf5-mpi-dev && export HDF5_MPI="ON" && pip install --no-binary=h5py h5py').
1. The libqcu.so was compiled when pyqcu setup in download_path/PyQCU/lib, please add this path to your LD_LIBRARY_PATH.
2. The QCU(PyQCU) splite grid by x->y->z->t, lattice by x->y->z->t->p->d->c->c or x->y->z->t->c->s(->p) and x->y->z->t->c->s->c->s(->p).
3. The QUDA(PyQUDA) splite grid by t->z->y->x, lattice by c->c->x->y->z->t->p->d or c->s->x->y->z->t(->p) and c->s->c->s->x->y->z->t(->p).
4. The QCU input params in numpy array(dtype=np.int32), argv in numpy array(dtype=np.float32 or float64) array, set_ptrs in numpy array(dtype=np.int64), other in cupy array(dtype=cp.complex64 or complex128).
5. The smallest lattice size is (x=4,y=4,z=4,t=8) that QCU support.

#####QCU NOTES END#####

Performance

The performance and accuracy comparison object of this project is the well-known benchmarking software [QUDA](#), but the main purpose is to learn from [QUDA](#), and the performance comparison is only for entertainment and reference.

→ **p2p, tuning, and multi-precision are turned off.**

```
23  ## quda
24  export QUDA_ENABLE_P2P=0
25  export QUDA_ENABLE_TUNING=0

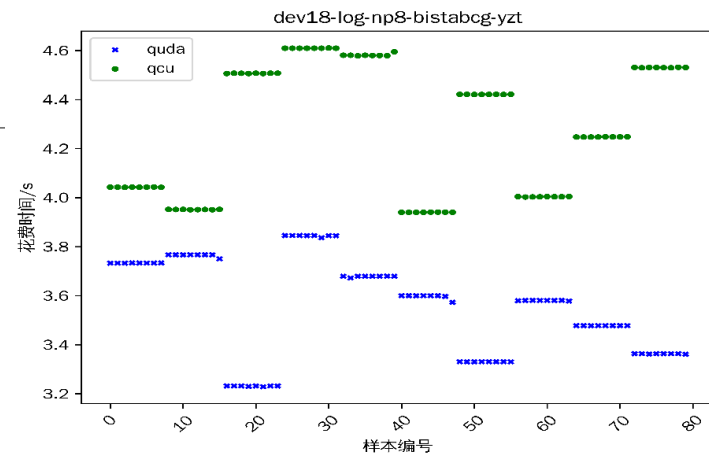
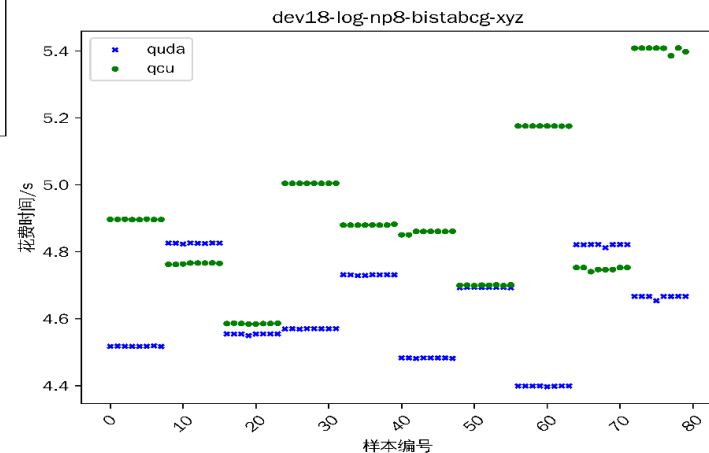
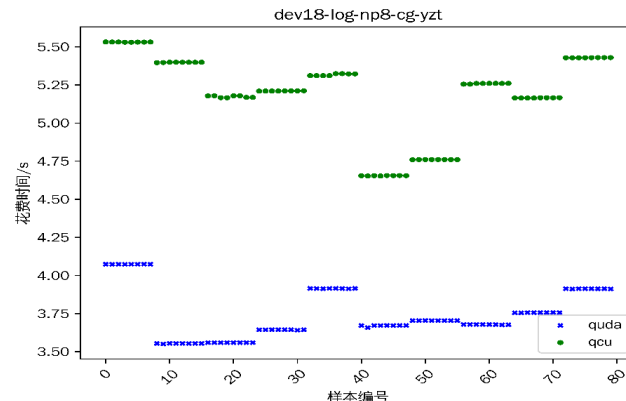
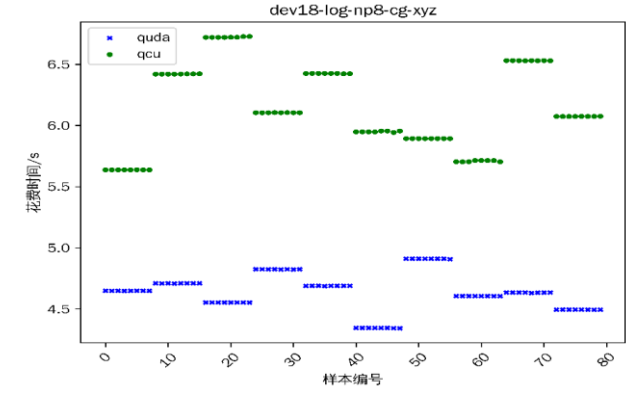
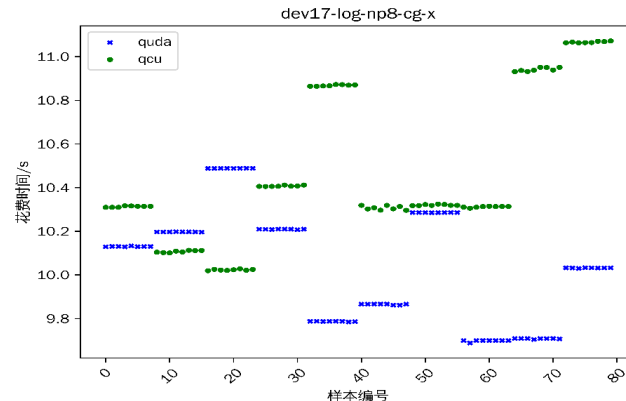
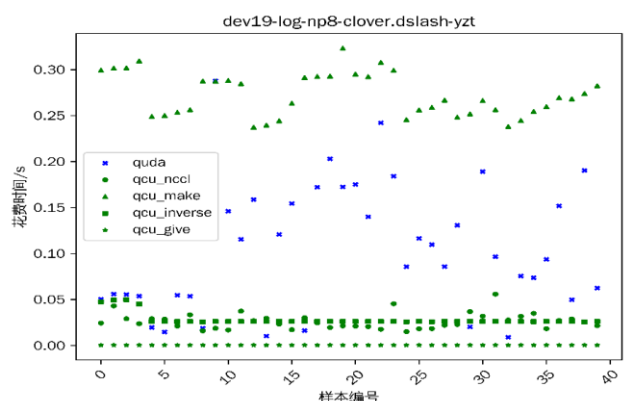
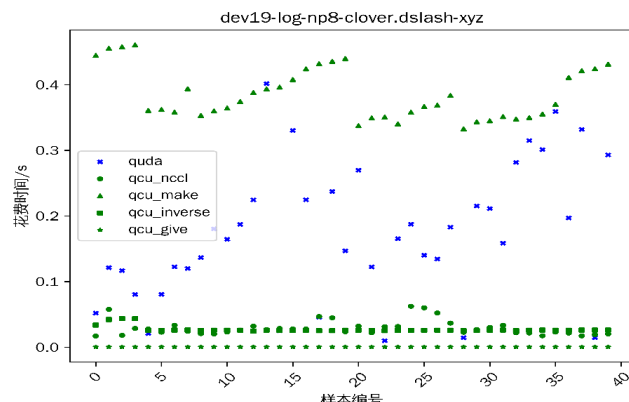
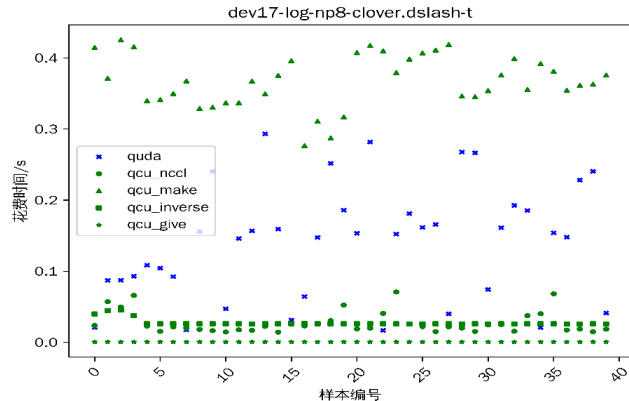
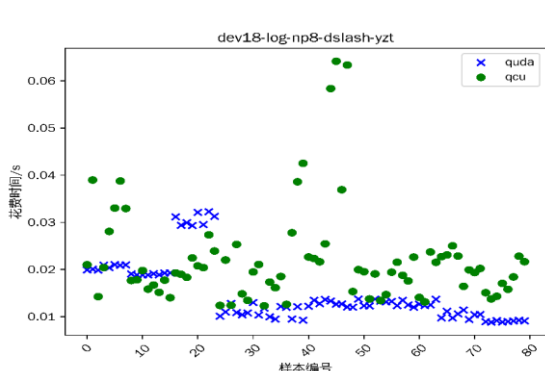
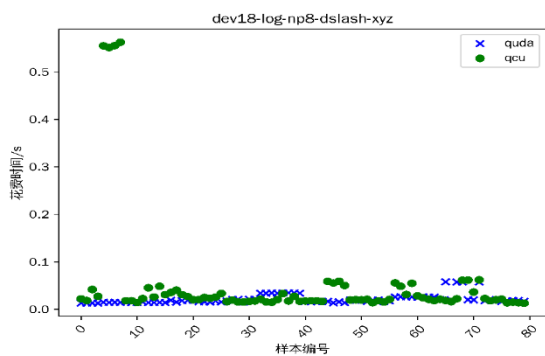
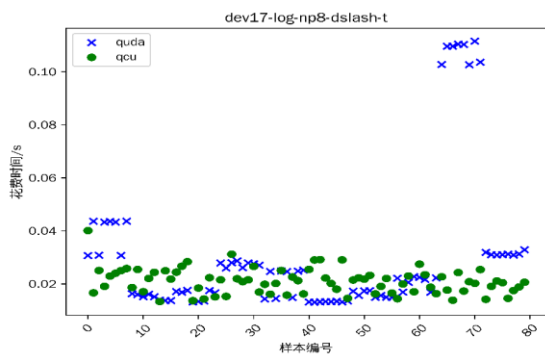
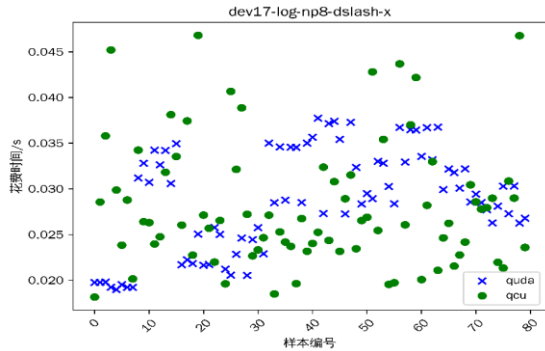
81  cpu_prec = QudaPrecision.QUDA_DOUBLE_PRECISION
82  cuda_prec = QudaPrecision.QUDA_DOUBLE_PRECISION
83  cuda_prec_sloppy = QudaPrecision.QUDA_DOUBLE_PRECISION
84  cuda_prec_precondition = QudaPrecision.QUDA_DOUBLE_PRECISION
85  cuda_prec_eigensolver = QudaPrecision.QUDA_DOUBLE_PRECISION
86  link_recon = QudaReconstructType.QUDA_RECONSTRUCT_12
87  link_recon_sloppy = QudaReconstructType.QUDA_RECONSTRUCT_12
88  ""
89  # changed by zx
90  cpu_prec = QudaPrecision.QUDA_DOUBLE_PRECISION
91  cuda_prec = QudaPrecision.QUDA_DOUBLE_PRECISION
92  cuda_prec_sloppy = QudaPrecision.QUDA_HALF_PRECISION
93  cuda_prec_precondition = QudaPrecision.QUDA_HALF_PRECISION
94  cuda_prec_eigensolver = QudaPrecision.QUDA_SINGLE_PRECISION
95  link_recon = QudaReconstructType.QUDA_RECONSTRUCT_12
96  link_recon_sloppy = QudaReconstructType.QUDA_RECONSTRUCT_12
97  ""

477  Creating Gaussian distributed Lie group field with sigma = 1.000000e-01
478  Quda dslash: 0.02001788793131709 sec
479  Quda dslash: 0.05775741802062839 sec
480  Quda dslash: 0.019795104046352208 sec
481  Quda dslash: 0.05730780097655952 sec
482  Quda dslash: 0.05741038604173809 sec
483  Quda dslash: 0.020204298896715045 sec
484  Quda dslash: 0.019900132901966572 sec
485  Quda dslash: 0.05747900204733014 sec
486  nccl wilson dslash total time: (without malloc free memcpy) :0.009534433 sec
487  nccl wilson dslash total time: (without malloc free memcpy) :0.011773425 sec
488  nccl wilson dslash total time: (without malloc free memcpy) :0.006994531 sec
489  nccl wilson dslash total time: (without malloc free memcpy) :0.011435001 sec
490  nccl wilson dslash total time: (without malloc free memcpy) :0.006869161 sec
491  nccl wilson dslash total time: (without malloc free memcpy) :0.009345476 sec
492  nccl wilson dslash total time: (without malloc free memcpy) :0.007942470 sec
493  nccl wilson dslash total time: (without malloc free memcpy) :0.014331865 sec
494  nccl wilson dslash total time: (without malloc free memcpy) :0.028961784 sec
495  nccl wilson dslash total time: (without malloc free memcpy) :0.032224457 sec
496  nccl wilson dslash total time: (without malloc free memcpy) :0.029139879 sec
497  nccl wilson dslash total time: (without malloc free memcpy) :0.032598733 sec
498  nccl wilson dslash total time: (without malloc free memcpy) :0.028939991 sec
499  nccl wilson dslash total time: (without malloc free memcpy) :0.007439768 sec
500  nccl wilson dslash total time: (without malloc free memcpy) :0.029809945 sec
501  nccl wilson dslash total time: (without malloc free memcpy) :0.032266250 sec
502  QCU dslash: 11.408965229056776 sec
503  quda difference: 2.805255728220984e-16
```

```
463  =====quda=====
464  CG: Convergence at 220 iterations, L2 relative residual: iterated = 9.770727e-10, true = 9.770727e-10 (requested = 1.000000e-09)
465  rank 0 quda x and x difference: , 4.234206183996702e-10, takes 4.554964599083178 sec, norm_quda_x = 195019.49423543355
466  quda rank 0 takes 4.554964599083178 sec
467  =====qcu=====
468  rank 0 quda x and x difference: , 4.206907575167709e-10, takes 4.554705031099729 sec, norm_quda_x = 195062.50910113816
469  quda rank 0 takes 4.554705031099729 sec
470  =====qcu=====
471  rank 0 quda x and x difference: , 4.3093730981883625e-10, takes 4.554749569972046 sec, norm_quda_x = 195191.4155583714
472  quda rank 0 takes 4.554749569972046 sec
473  =====qcu=====
474  rank 0 quda x and x difference: , 4.24892220436779e-10, takes 4.549927397049032 sec, norm_quda_x = 195259.17141863785
475  quda rank 0 takes 4.549927397049032 sec
476  =====qcu=====
477  rank 0 quda x and x difference: , 4.461352176705238e-10, takes 4.554710172000341 sec, norm_quda_x = 195248.34425321454
478  quda rank 0 takes 4.554710172000341 sec
479  =====qcu=====
480  rank 0 quda x and x difference: , 4.3579659517008474e-10, takes 4.5549612829927355 sec, norm_quda_x = 195221.36519752583
481  quda rank 0 takes 4.5549612829927355 sec
482  =====qcu=====
483  rank 0 quda x and x difference: , 4.256779755212629e-10, takes 4.555050499970093 sec, norm_quda_x = 195292.60902476698
484  quda rank 0 takes 4.555050499970093 sec
485  =====qcu=====
486  rank 0 quda x and x difference: , 4.2665474454781144e-10, takes 4.554805380990729 sec, norm_quda_x = 195218.25935830487
487  quda rank 0 takes 4.554805380990729 sec
488  =====qcu=====
489  ##RANK:7##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
490  nccl wilson bistabcg total time: (without malloc free memcpy) :4.585736828 sec
491  ##RANK:5##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
492  nccl wilson bistabcg total time: (without malloc free memcpy) :4.586742864 sec
493  ##RANK:3##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
494  nccl wilson bistabcg total time: (without malloc free memcpy) :4.586113890 sec
495  ##RANK:0##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
496  nccl wilson bistabcg total time: (without malloc free memcpy) :4.584611544 sec
497  ##RANK:1##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
498  nccl wilson bistabcg total time: (without malloc free memcpy) :4.584584912 sec
499  ##RANK:6##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
500  nccl wilson bistabcg total time: (without malloc free memcpy) :4.585700756 sec
501  ##RANK:2##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
502  nccl wilson bistabcg total time: (without malloc free memcpy) :4.586126475 sec
503  ##RANK:4##LOOP:120##Residual:(7.27889e-10,-2.8944e-30i)
504  nccl wilson bistabcg total time: (without malloc free memcpy) :4.586732934 sec
```

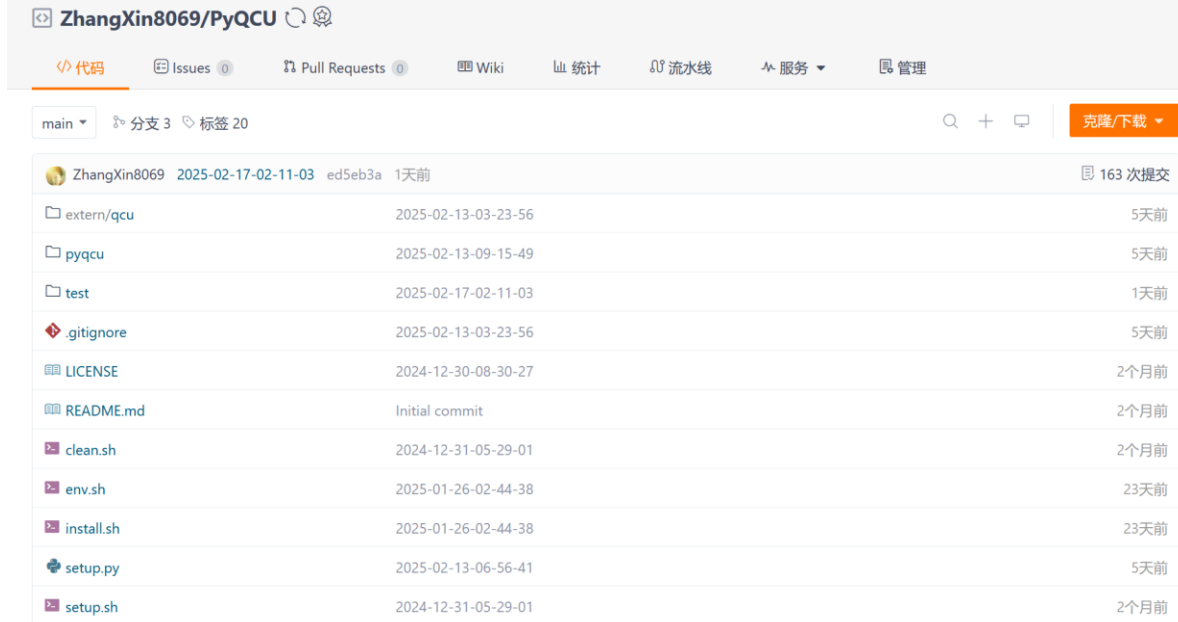
Performance

The performance and accuracy comparison object of this project is the well-known benchmarking software [QUDA](#), but the main purpose is to learn from [QUDA](#), and the performance comparison is only for entertainment and reference.



PyQCU

```
1 import re
2 import numpy as np
3 import cupy as cp
4 from pyqcu import define
5 from pyqcu import io
6 from pyqcu import qcu
7 from pyqcu.set import params, argv, set_ptrs
8 import h5py
9 print('My rank is ', define.rank)
10 params[define._LAT_Y_] /= 2
11 params[define._LAT_XYZT_] /= 2
12 params[define._SET_PLAN_] = 2
13 gauge_filename = f"quda_wilson-clover-dslash-gauge_{params[define._LAT_X_]}-{params[define._LAT_Y_]}"
14 params[define._GRID_T_] = 8
15 params[define._NODE_RANK_] = define.rank
16 params[define._NODE_SIZE_] = define.size
17 print("Parameters:", params)
18 #####
19 print("Gauge filename:", gauge_filename)
20 gauge = io.hdf5_xxxtzyx2grid_xxxtzyx(params, gauge_filename)
21 fermion_in_filename = gauge_filename.replace("gauge", "fermion-in")
22 print("Fermion in filename:", fermion_in_filename)
23 fermion_in = io.hdf5_xxxtzyx2grid_xxxtzyx(params, fermion_in_filename)
24 fermion_out_filename = gauge_filename.replace("gauge", "fermion-out")
25 print("Fermion out filename:", fermion_out_filename)
26 quda_fermion_out = io.hdf5_xxxtzyx2grid_xxxtzyx(params, fermion_out_filename)
27 #####
28 fermion_out = cp.zeros_like(fermion_in)
29 print("Fermion out data:", fermion_out.data)
30 print("Fermion out shape:", fermion_out.shape)
31 #####
32 qcu.applyInitQcu(set_ptrs, params, argv)
33 # qcu.applyWilsonDslashQcu(fermion_out, fermion_in, gauge, set_ptrs, params)
34 qcu.applyCloverDslashQcu(fermion_out, fermion_in, gauge, set_ptrs, params)
35 qcu.applyEndQcu(set_ptrs, params)
36 #####
37 print("Fermion out data:", fermion_out.data)
38 print("Fermion out shape:", fermion_out.shape)
39 print("QUDA Fermion out data:", quda_fermion_out.data)
40 print("QUDA Fermion out shape:", quda_fermion_out.shape)
41 print("Difference:", cp.linalg.norm(fermion_out -
42     quda_fermion_out)/cp.linalg.norm(quda_fermion_out))
43 #####
44 io.grid_xxxtzyx2hdf5_xxxtzyx(fermion_out, params)
```



Just 'git clone <https://gitee.com/zhangxin8069/PyQCU.git>',
then 'bash ./install.sh' in there.
PyQCU is ready for you!

PyQCU

考虑到未来的用户上手Linux可能有一定的困难（主要是系统环境配置），在此提供了开箱即用的Docker镜像：

The screenshot shows the Docker Hub search results for the user 'zhangxin8069'. The interface includes a blue header with the Docker Hub logo, a search bar containing 'zhangxin8069', and navigation links for 'Sign in' and 'Sign up'. Below the header, the search results are displayed, showing 1 - 5 of 5 results. The results are filtered by 'Products' (Images, Extensions, Plugins) and 'Trusted content' (Docker Official Image, Verified Publisher, Sponsored OSS). The categories listed are API Management, Content Management System, Data Science, Databases & Storage, and Languages & Frameworks. The search results list five Docker images:

- zhangxin8069/x99-v20241023-image**: By [zhangxin8069](#) · Updated 4 months ago. Downloads: 16, Stars: 0.
- zhangxin8069/v20241023-image**: By [zhangxin8069](#) · Updated 5 months ago. Downloads: 13, Stars: 0.
- zhangxin8069/x99-v20250220**: By [zhangxin8069](#) · Updated 20 days ago. Downloads: 7, Stars: 0.
- zhangxin8069/computer-v20250216**: By [zhangxin8069](#) · Updated 23 days ago. Downloads: 7, Stars: 0.
- zhangxin8069/computer-v20241023-image**: By [zhangxin8069](#) · Updated 4 months ago. Downloads: 1, Stars: 0.

A vertical 'Give Feedback' button is visible on the right side of the search results.

**我的汇报到此结束
谢谢大家!**